

Digital Image Processing Lab01 Report

EECS26 111060024 蔡孟伶

Project02_02

Implementation

This program adjusts the input image's intensity level, which is taken as the variable input, ranging from 2 to 256, and could only be integer powers of 2. The algorithm is quite straightforward in that we first divide the pixel value into different intensity levels. For example, if the output image should have two colors (black and white), then the pixels whose value is bigger than 127 would be white, otherwise black. If the image should have 16 intensity levels, then for each level, there are $256 / 16 = 16$ kinds of possibilities. Such calculation may result in floating / double numbers, I then estimate it to be integer by the rounding function.

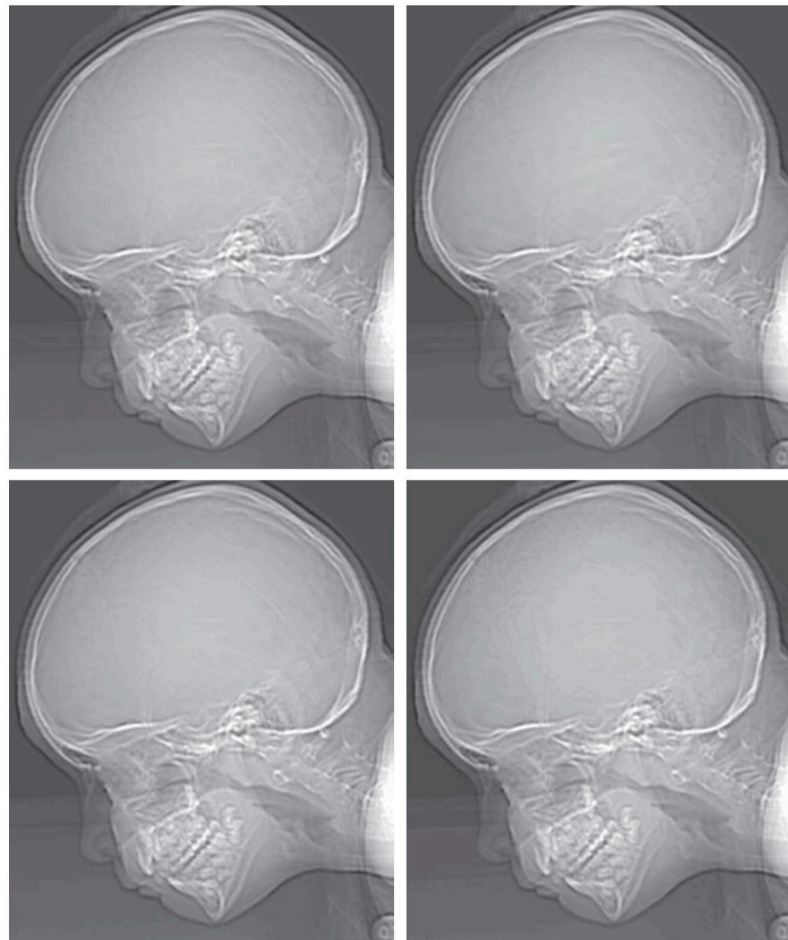
In the function I've designed, process could be done by mathematical operation, thus, I didn't generate a blank new grid / matrix to be the quantized image, instead, I directly calculated the output and showed it out by imshow function. The input image is read by the program via imread function and relative path of the image, additively.

Result Image

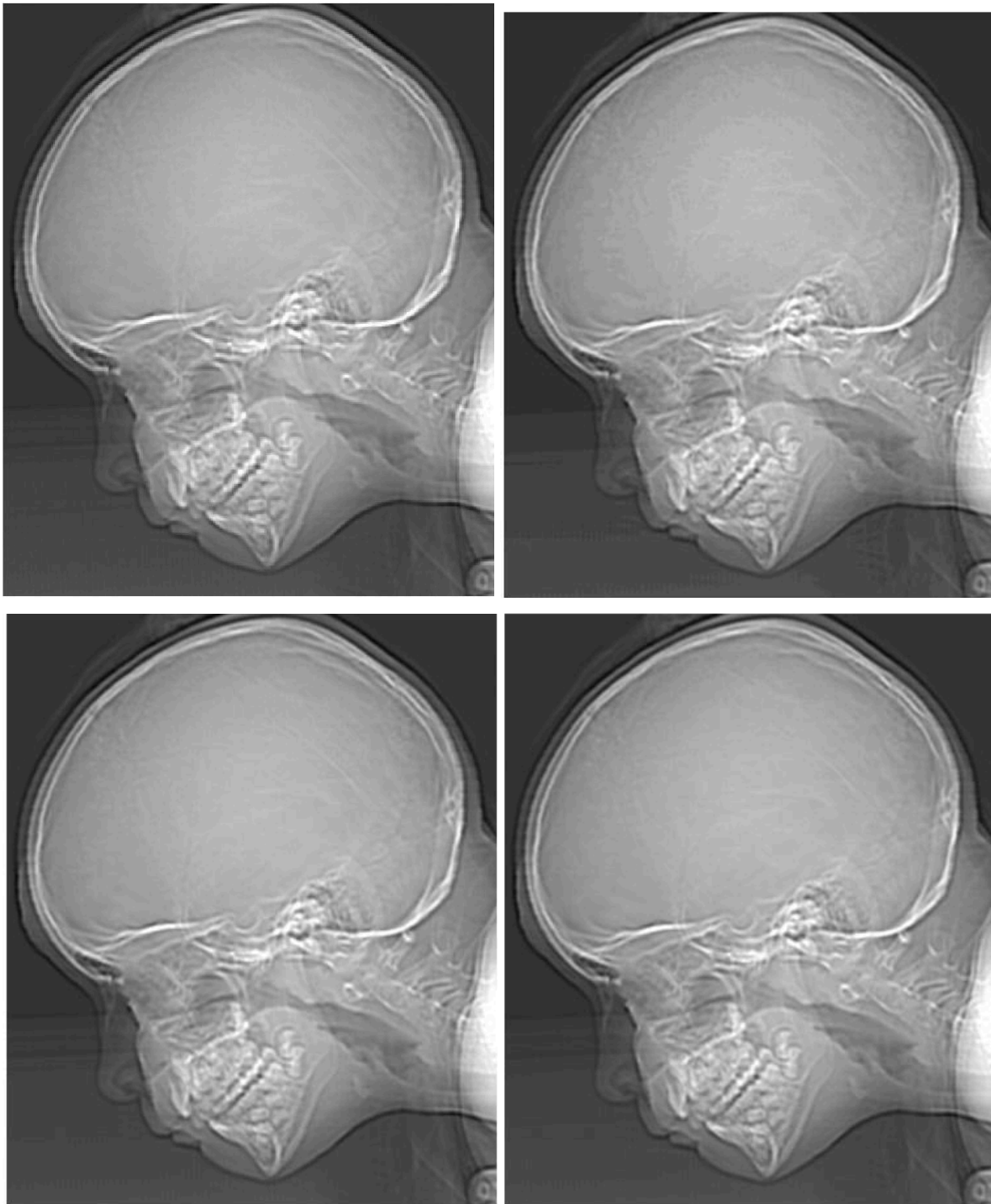
a	b
c	d

FIGURE 2.24

(a) 774×640 , 256-level image.
(b)-(d) Image displayed in 128, 64, and 32 intensity levels, while keeping the spatial resolution constant.
(Original image courtesy of the Dr. David R. Pickens, Department of Radiology & Radiological Sciences, Vanderbilt University Medical Center.)

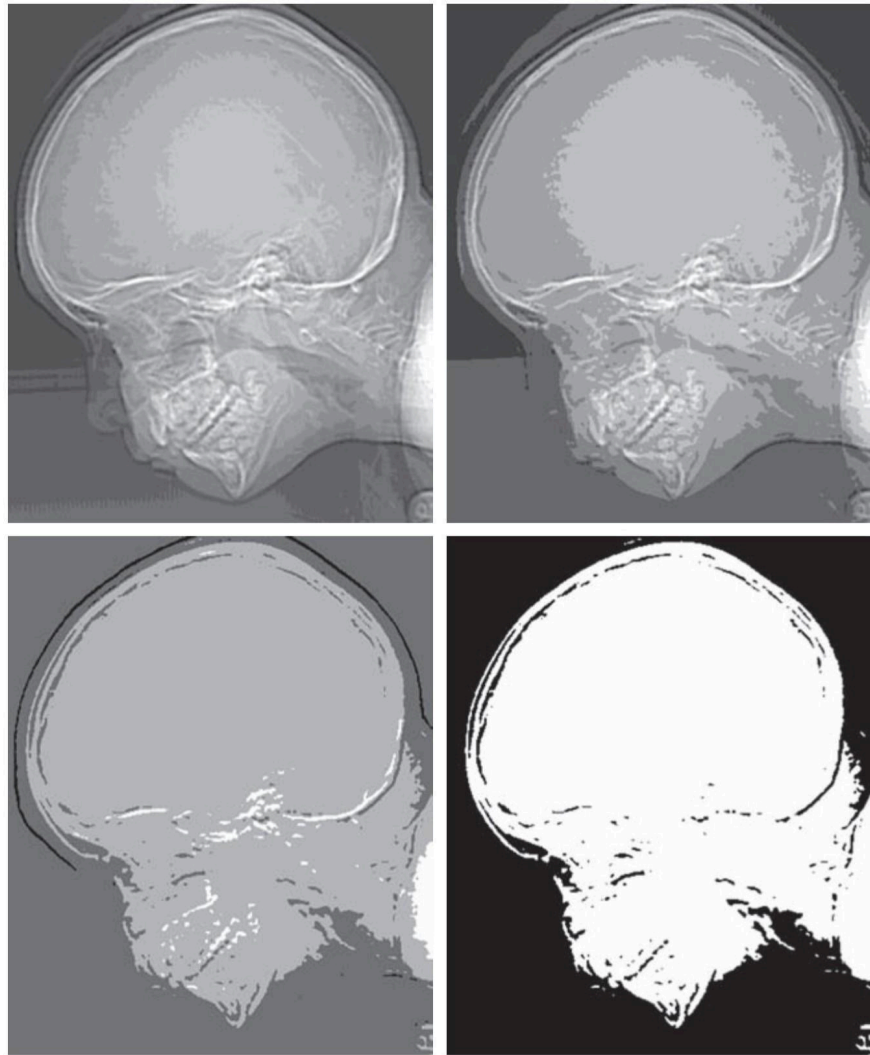


Following is the result image after the input picture processed by `reduceIntensityLevel`, with intensity level = 256, 128, 64, 32 from left top to right bottom.

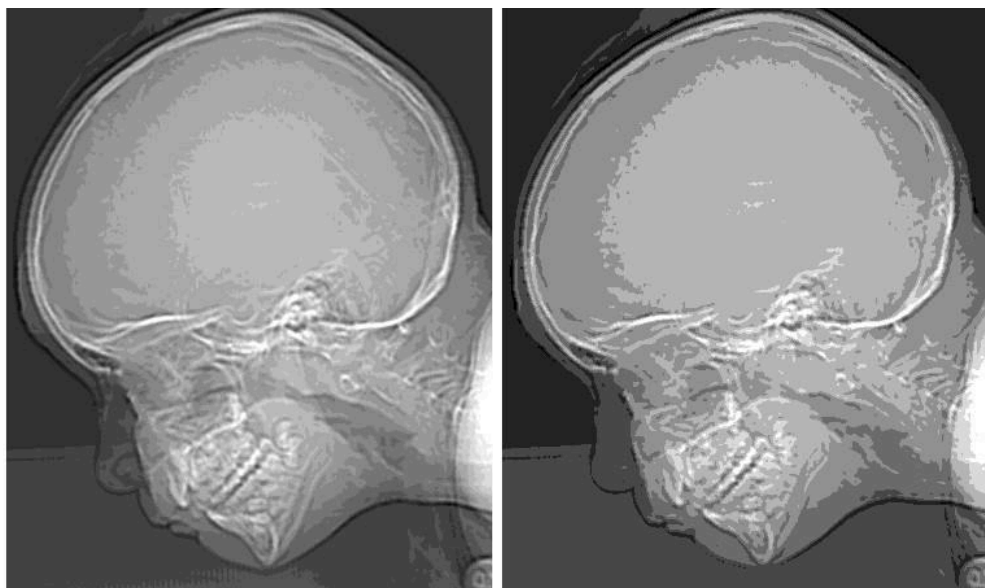


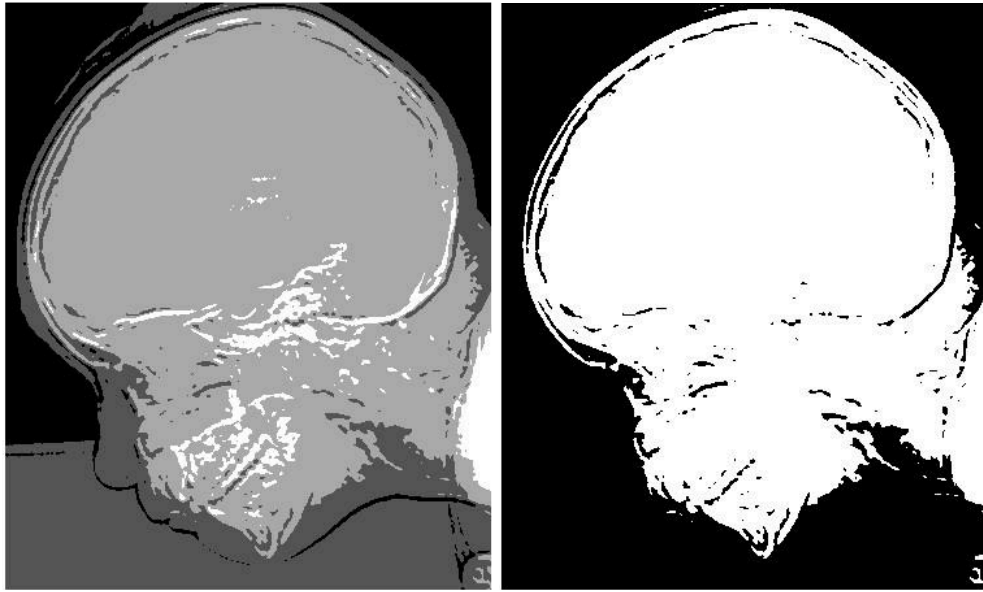
e	f
g	h

FIGURE 2.24
(Continued)
(e)-(h) Image
displayed in 16, 8,
4, and 2 intensity
levels.



Following is the result image after the input picture processed by `reduceIntensityLevel`, with intensity level = 16, 8, 4, 2 from left top to right bottom.





Projecr02_03

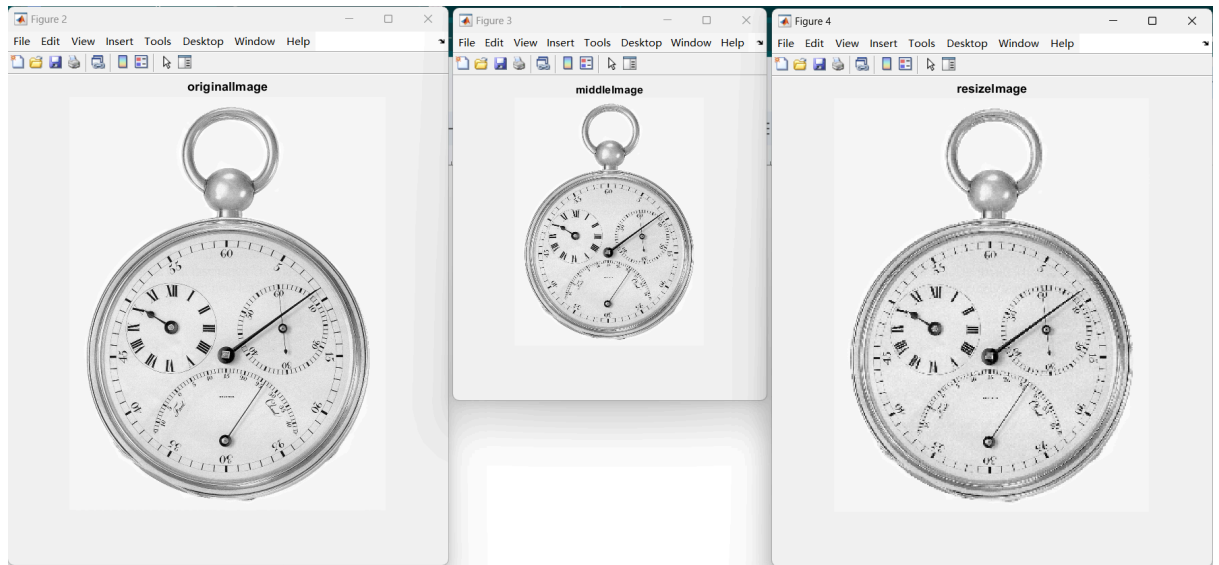
Implementation

This function implements the resize function (i.e. both shrinking and zooming) by pixel replication. In fact, it went through the same logic no matter if it's shrinking or zooming. First, I calculated the number of rows and columns after resizing, then for each new pixel on the grid, I sampled the point from the original image. If the image is shrunked, the sample point skipped equal pixels each time. If the image is zoomed, it takes the neighboring pixel's value to fill in the pixel which doesn't exist in the original image. This is the concept of pixel replication.

It is worth noting that I used double for loop to implement this function at first, but it turned out the time efficiency is too low. Since MATLAB deals with the manipulation of matrices well, it is better to approach the answer by the index vectors. This kind of implementation avoids redundant calculations and improves efficiency.

Result Image

From left to right is the original image, the processed image that shrunked by a factor of 10, and the resized image that zoomed it to the original size. Considering the layout of the report, it's hard to show the image both by detail and proper size. I think the best way to get the result is to run the program.



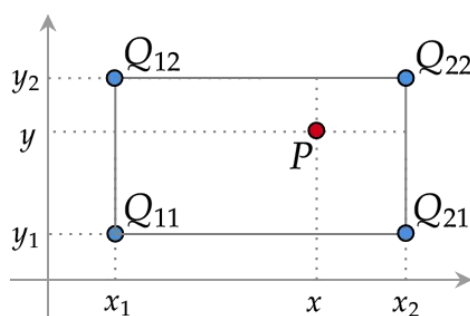
Discussion

After shrinking the image then zooming back again, the image looked unclear. This is because when it shrinks to the 1/10 of the original size, it takes 1/10 sample points on each row and column. When it comes to enlarging it back, all the program does is just copying pixel values to make the picture bigger, the DPI of the image doesn't improve.

Project02_04

Implementation

This function, same as previous, is supposed to resize the input image. However, in this function, the data is processed within more detail that when it comes to zooming, most pixel value are calculated by bilinear interpolation formula. We first calculate the original sample point that each new pixel mapped to, then take the four neighboring pixels of that. Within these four pixel value, delta x, and delta y, apply each pixel to bilinear interpolation formula gives a result different from merely pixel replication. It kind of smoothes the relationship of different sample pixel values.



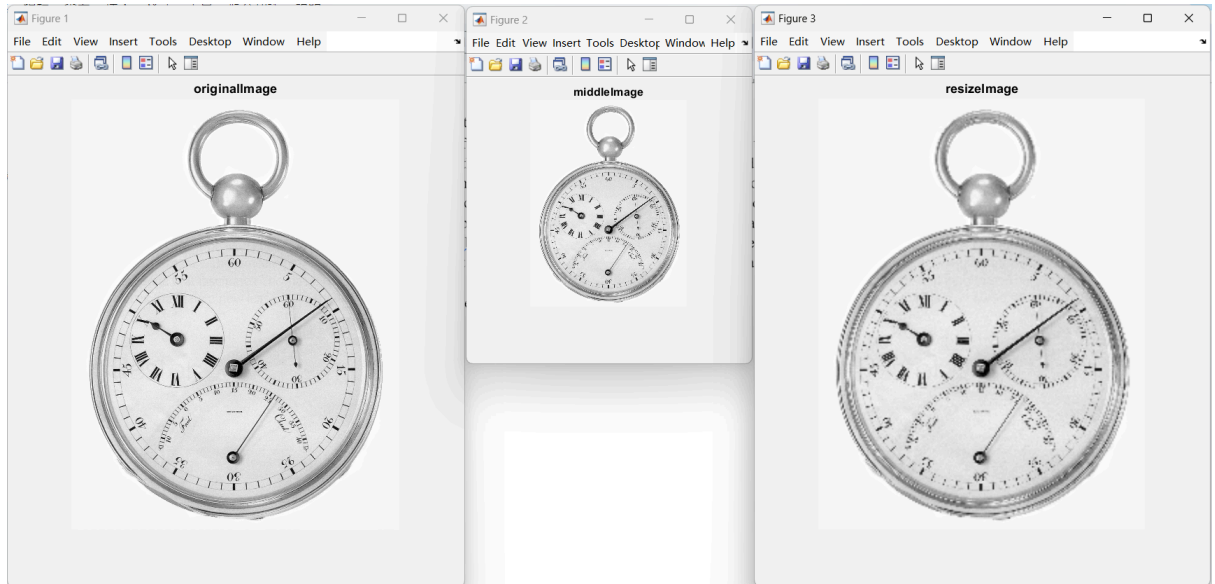
To explain the formula of bilinear interpolation, for every pixel of the new image, I calculate it by new_x and new_y divided by scalingFactor and make sure it doesn't get out of the bound. It's clear to look at the following picture. Suppose P is the original point I've mapped (it's normal that it has floating coordinates). I access the pixel value of Q11, Q12, Q21, Q22 of the original image, and calculate delta x and y, which is $x - x_1$ and $y - y_1$. In the formula of

bilinear interpolation, it should be

$$\text{resizedImage}(\text{new_x}, \text{new_y}) = (1 - \text{delta_x}) * (1 - \text{delta_y}) * Q_{11} + (1 - \text{delta_x}) * \text{delta_y} * Q_{12} + \text{delta_x} * (1 - \text{delta_y}) * Q_{21} + \text{delta_x} * \text{delta_y} * Q_{22}.$$

Result Image

from left to right is the original image, the image that is processed with scaling factor = 0.08, and finally back to original size.



Discussion

The output image will not be the same as the input image although it is shrunk then zoomed back to the original size. This is because that when the image is shrunk, it takes only some sample points on the original image to fit the size. However, when it comes to zooming, since we're using bilinear interpolation, it is impossible for every pixel, calculated, to have the exact same value as the original one. You can tell that the edge is kind of blurry, and it just doesn't look as clear as before.

When you zoom in the middleImage and get back to the original size, you can see that the edge of the lines are kind of blurry, this is one of the evidence that bilinear interpolation is used. If you compare the resized image with the previous project with a closer look, you can tell the difference. Pixel replication actually have the exact same value nearby, but the bilinear interpolation one has it changed progressively.

